- **Embedded Firmware Design and Development**
  - Embedded Firmware Design Approaches
    - *Super loop* based approach
    - *Embedded Operating System* based approach
  - Embedded Firmware Development Languages
    - *Assembly Language*
    - *High Level Language*

Embedded Firmware Design & Development

- ✓The embedded firmware is responsible for controlling the various peripherals of the embedded hardware and generating response in accordance with the functional requirements of the product.
- ✓The embedded firmware is usually stored in a permanent memory (ROM) and it is non alterable by end users.
- ✓Designing Embedded firmware requires understanding of the particular embedded product hardware, like various component interfacing, memory map details, I/O port details, configuration and register details of various hardware chips used and some programming language (either low level Assembly Language or High level language like C/C++ or a combination of the two).

Embedded Firmware Design & Development

- ✓The embedded firmware development process starts with the conversion of the firmware requirements into a program model using various modeling tools
- ✓There exist two basic approaches for the design and implementation of embedded firmware, namely;
  - ✓The *Super loop* based approach
  - ✓The *Embedded Operating System* based approach
- ✓The decision on which approach needs to be adopted for firmware development is purely dependent on the complexity and system requirements

Embedded Firmware Design & Development

Embedded firmware Design Approaches – The Super loop

- ✓ Suitable for applications that are not time critical and where the response time is not so important (Embedded systems where missing deadlines are acceptable)
- ✓ Very similar to a conventional procedural programming where the code is executed task by task
- ✓ The tasks are executed in a never ending loop. The task listed on top on the program code is executed first and the tasks just below the top are executed after completing the first task

Embedded Firmware Design & Development

Embedded firmware Design Approaches – The Super loop

✓ A typical super loop implementation will look like:

1Configure the common parameters and perform initialization for various hardware componen memory, registers etc.

2Start the first task and execute it

3Execute the second task

4Execute the next task

5:

6:

7Execute the last defined task

8Jump back to the first task and follow the same flow

```
void main ()
{
Configurations ();
Initializations ();
while (1)
    {
       Task 1 ();
       Task 2 ();
       //:
       //:
       Task n ();
    }
}
```

Embedded Firmware Design & Development

Embedded firmware Design Approaches – The Super loop

Pros:

- ✓Doesn't require an Operating System for task scheduling and monitoring and free from OS related overheads
- ✓Simple and straight forward design
- ✓Reduced memory footprint

Cons:

- ✓Non Real time in execution behavior (As the number of tasks increases the frequency at which a task gets CPU time for execution also increases)
- ✓Any issues in any task execution may affect the functioning of the product (This can be effectively tackled by using Watch Dog Timers for task execution monitoring)

Embedded Firmware Design & Development

Embedded firmware Design Approaches – The Super loop

Enhancements:

- ✓Combine Super loop based technique with interrupts
- ✓Execute the tasks (like keyboard handling) which require Real time attention as Interrupt Service routines

Embedded Firmware Design & Development

Embedded firmware Design Approaches – Embedded OS based Approach

- ✓ The embedded device contains an Embedded Operating System which can be one of:
- ✓ A Real Time Operating System (RTOS)
- ✓ A Customized General Purpose Operating System (GPOS)
- ✓ The Embedded OS is responsible for scheduling the execution of user tasks and the allocation of system resources among multiple tasks
- ✓ Involves lot of OS related overheads apart from managing and executing user defined tasks

Embedded Firmware Design & Development

Embedded firmware Design Approaches – Embedded OS based Approach

- ✓ Microsoft® Windows XP Embedded is an example of GPOS for embedded devices
- ✓ Point of Sale (PoS) terminals, Gaming Stations, Tablet PCs etc are examples of embedded devices running on embedded GPOSs
- ✓ '*Windows CE*', '*Windows Mobile*','*QNX*', '*VxWorks*', '*ThreadX*', '*MicroC/OS-II*', '*Embedded Linux*', '*Symbian*' etc are examples of RTOSs employed in Embedded Product development
- ✓ Mobile Phones, PDAs, Flight Control Systems etc are examples of embedded devices that runs on RTOSs

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options

- ✓Assembly Language
- ✓High Level Language
  - ✓Subset of C (Embedded C)
  - ✓Subset of C++ (Embedded C++)
  - ✓Any other high level language with supported Cross-compiler
- ✓Mix of Assembly & High level Language
  - ✓Mixing High Level Language (Like C) with Assembly Code
  - ✓Mixing Assembly code with High Level Language (Like C)
  - ✓Inline Assembly

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

- ✓ '***Assembly Language***' is the human readable notation of '***machine language***'
- ✓ '***machine language***' is a processor understandable language
- ✓ Machine language is a binary representation and it consists of 1s and 0s
- ✓ Assembly language and machine languages are processor/controller dependent
- ✓ An Assembly language program written for one processor/controller family will not work with others

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

- ✓ **Assembly language programming is the process of writing processor specific machine code in mnemonic form, converting the mnemonics into actual processor instructions (machine language) and associated data using an assembler**
- ✓ The general format of an assembly language instruction is an Opcode followed by Operands
- ✓ The Opcode tells the processor/controller what to do and the Operands provide the data and information required to perform the action specified by the opcode
- ✓ It is not necessary that all opcode should have Operands following them. Some of the Opcode implicitly contains the operand and in such situation no operand is required. The operand may be a single operand, dual operand or more

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

The 8051 Assembly Instruction

MOV A, #30

Moves decimal value 30 to the 8051 Accumulator register. Here *MOV A* is the Opcode and 30 is the operand (single operand). The same instruction when written in machine language will look like

01110100 00011110

The first 8 bit binary value 01110100 represents the opcode *MOV A* and the second 8 bit binary value 00011110 represents the operand 30.

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

- ✓ Assembly language instructions are written one per line
- ✓ A machine code program consists of a sequence of assembly language instructions, where each statement contains a mnemonic (Opcode + Operand)
- ✓ Each line of an assembly language program is split into four fields as:

LABEL OPCODE OPERAND COMMENTS

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

- ✓ LABEL is an optional field. A 'LABEL' is an identifier used extensively in programs to reduce the reliance on programmers for remembering where data or code is located. LABEL is commonly used for representing
- ✓A memory location, address of a program, sub-routine, code portion etc.
- ✓The maximum length of a label differs between assemblers. Assemblers insist strict formats for labeling. Labels are always suffixed by a colon and begin with a valid character. Labels can contain number from 0 to 9 and special character _ (underscore).

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

```
;#########################################################################
##
;           SUBROUTINE FOR GENERATING DELAY
;           DELAY PARAMETR PASSED THROUGH REGISTER R1
;           RETURN VALUE NONE
;           REGISTERS USED: R0,  R1
;#########################################################################
##          DELAY:  MOV     R0, #255 ; Load Register R0 with 255
                    DJNZ    R1, DELAY          ; Decrement R1 and loop till
          R1= 0
                    RET                        ; Return to calling program
```

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language

- ✓The symbol ; represents the start of a comment. Assembler ignores the text in a line after the ; symbol while assembling the program
- ✓DELAY is a label for representing the start address of the memory location where the piece of code is located in code memory
- ✓The above piece of code can be executed by giving the label DELAY as part of the instruction. E.g. LCALL DELAY; LMP DELAY

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language – Source File to Hex File Translation

- ✓The Assembly language program written in assembly code is saved as *.asm* (Assembly file) file or a *.src* (source) file or a format supported by the assembler
- ✓Similar to 'C' and other high level language programming, it is possible to have multiple source files called modules in assembly language programming. Each module is represented by a '***.asm***' or '***.src***' file or the assembler supported file format similar to the '***.c***' files in C programming
- ✓The software utility called 'Assembler' performs the translation of assembly code to machine code

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language – Source File to Hex File Translation

- ✓The assemblers for different family of target machines are different. A51 Macro Assembler from Keil software is a popular assembler for the 8051 family micro controller
- ✓Each source file can be assembled separately to examine the syntax errors and incorrect assembly instructions
- ✓Assembling of each source file generates a corresponding object file. The object file does not contain the absolute address of where the generated code needs to be placed (a re-locatable code) on the program memory

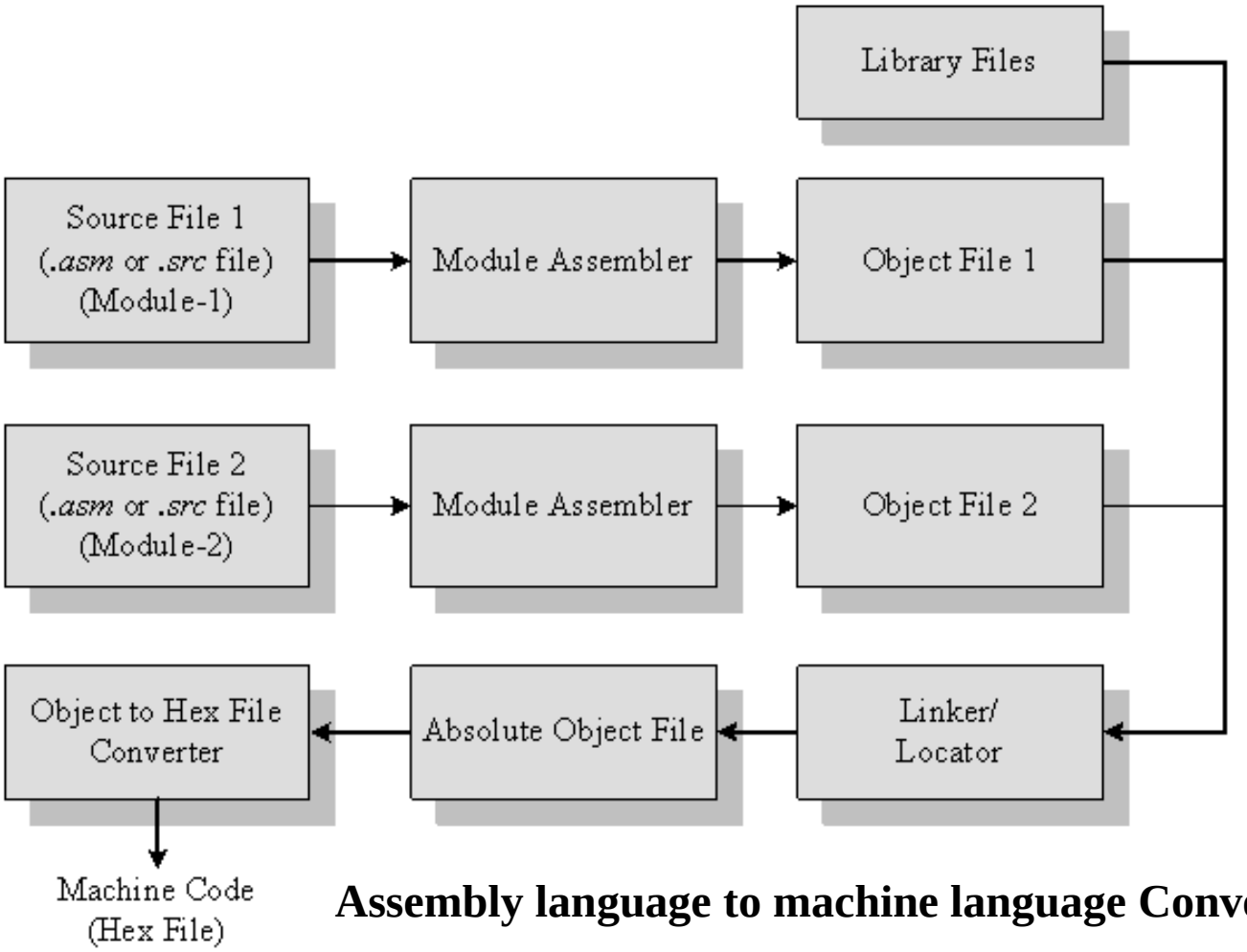
Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language – Source File to Hex File Translation

- ✓The software program called linker/locater is responsible for assigning absolute address to object files during the linking process
- ✓The Absolute object file created from the object files corresponding to different source code modules contain information about the address where each instruction needs to be placed in code memory
- ✓A software utility called ‘Object to Hex file converter’ translates the absolute object file to corresponding hex file (binary file)

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language – Source File to Hex File Translation



Assembly language to machine language Conversion process

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Assembly Language – Source File to Hex File Translation

Advantages:

- ✓Efficient Code Memory & Data Memory Usage (Memory Optimization)
- ✓High Performance
- ✓Low level Hardware Access
- ✓Code Reverse Engineering

Drawbacks:

- ✓High Development time
- ✓Developer dependency
- ✓Non portable

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – High Level Language

- ✓ The embedded firmware is written in any high level language like C, C++
- ✓ A software utility called ‘cross-compiler’ converts the high level language to target processor specific machine code
- ✓ The cross-compilation of each module generates a corresponding object file. The object file does not contain the absolute address of where the generated code needs to be placed (a re-locatable code) on the program memory

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – High Level Language

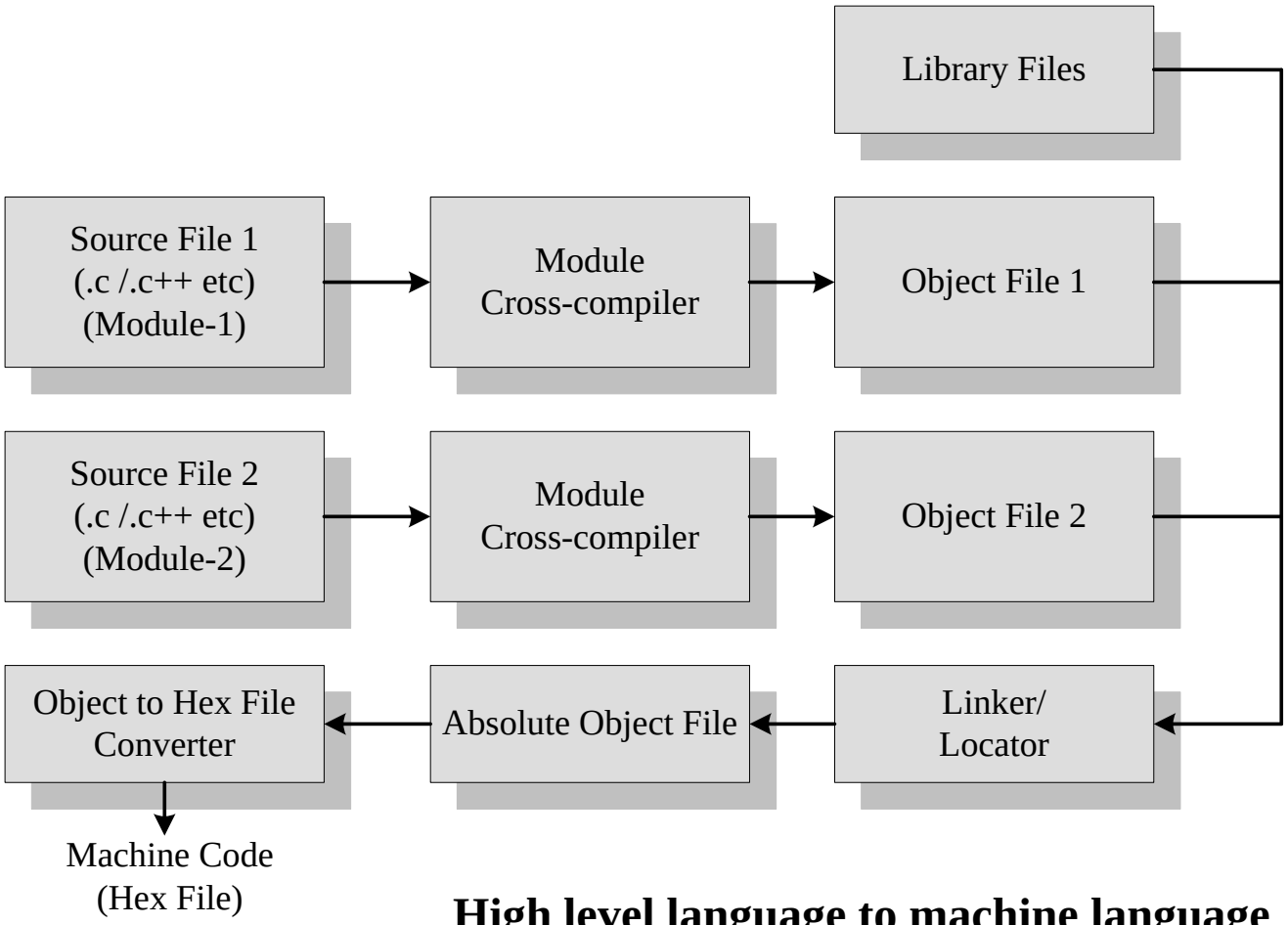
✓The software program called linker/locater is responsible for assigning absolute address to object files during the linking process

✓The Absolute object file created from the object files corresponding to different source code modules contain information about the address where each instruction needs to be placed in code memory

✓A software utility called ‘Object to Hex file converter’ translates the absolute object file to corresponding hex file (binary file)

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – High Level Language – Source File to Hex File Translation



High level language to machine language Conversion process

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – High Levely Language – Source File to Hex File Translation

Advantages:

- ✓Reduced Development time
- ✓Developer independency
- ✓Portability

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Mixing of Assembly Language with High Level Language

- ✓ Certain situations in Embedded firmware development may require the mixing of Assembly Language with high level language or vice versa. Interrupt handling, Source code is already available in high level language\Assembly Language etc are examples
- ✓ High Level language and low level language can be mixed in three different ways

Embedded Firmware Design & Development

Embedded firmware Development Languages/Options – Mixing of Assembly Language with High Level Language

- ✓Mixing Assembly Language with High level language like ‘C’
- ✓Mixing High level language like ‘C’ with Assembly Language
- ✓In line Assembly
- ✓ The passing of parameters and return values between the high level and low level language is cross-compiler specific

Embedded Firmware Design & Development

High Level Language – 'C' V/s Embedded C

- ✓ '*C*' is a well structured, well defined and standardized general purpose programming language with extensive bit manipulation support
- ✓ '*C*' offers a combination of the features of high level language and assembly and helps in hardware access programming (system level programming) as well as business package developments (Application developments like pay roll systems, banking applications etc)
- ✓ The conventional '*C*' language follows ANSI standard and it incorporates various library files for different operating systems
- ✓ A platform (Operating System) specific application, known as, compiler is used for the conversion of programs written in '*C*' to the target processor (on which the OS is running) specific binary files

Embedded Firmware Design & Development

High Level Language – 'C' V/s Embedded C

- ✓ Embedded C can be considered as a subset of conventional 'C' language
- ✓ Embedded C supports all 'C' instructions and incorporates a few target processor specific functions/instructions
- ✓ The standard ANSI 'C' library implementation is always tailored to the target processor/controller library files in Embedded C
- ✓ The implementation of target processor/controller specific functions/instructions depends upon the processor/controller as well as the supported cross-compiler for the particular Embedded C language
- ✓ A software program called 'Cross-compiler' is used for the conversion of programs written in Embedded C to target processor/controller specific instructions

Embedded Firmware Design & Development

High Level Language Based Development – 'Compiler' V/s 'Cross-Compiler'

- ✓ Compiler is a software tool that converts a source code written in a high level language on top of a particular operating system running on a specific target processor architecture (E.g. Intel x86/Pentium).
- ✓ The operating system, the compiler program and the application making use of the source code run on the same target processor.
- ✓ The source code is converted to the target processor specific machine instructions
- ✓ A native compiler generates machine code for the same machine (processor) on which it is running.
- ✓ Cross compiler is the software tools used in cross-platform development applications

Embedded Firmware Design & Development

High Level Language Based Development – 'Compiler' V/s 'Cross-Compiler'

- ✓ In cross-platform development, the compiler running on a particular target processor/OS converts the source code to machine code for a target processor whose architecture and instruction set is different from the processor on which the compiler is running or for an operating system which is different from the current development environment OS
- ✓ Embedded system development is a typical example for cross-platform development where embedded firmware is developed on a machine with Intel/AMD or any other target processors and the same is converted into machine code for any other target processor architecture (E.g. 8051, PIC, ARM9 etc).
- ✓ Keil C51compiler from Keil software is an example for cross-compiler for 8051 family architecture